Prime Scientific: moving purchase tracking to the server

Prepared 7 September 2026 for the Prime Scientific development team. Business `biz_k1QVwJt74iS6zP` / `primescientificlab.com`.

Short version. Purchase tracking on this store works, and the numbers are real: **33 orders worth 7,284.04 dollars** in the last ten days, each with the correct value attached. Nothing here is broken. The change being asked for is that the purchase event is fired by the buyer's browser, and it needs to be fired by your server instead, so that the orders the browser cannot report stop going missing.

1. What was measured, and how

Every figure in this document was read live from the Whop events API on 7 September 2026, over a 28 August 2026 to 7 September 2026 (ten days) window. The sweep followed the API's paging cursors to the end and returned **3,716 events across 8 pages**. A partial read was not accepted: the tool used refuses to write a result at all if any page fails, because presence survives a truncated list but **absence does not**.

> **One reading trap worth naming, because it silently produces a wrong answer.** On `GET /events` every custom event, purchases included, comes back with `event_name: "pixel.custom"`. The real name lives in a separate `custom_name` field. Filtering on `event_name` for "purchase" returns zero even on an account that provably has purchases. Every count below was taken across both fields.

2. Where purchases stand today

what	measured
Purchase events in the window	33
Total value recorded	7,284.04 USD
Events carrying a real order value	33 of 33
Pages they fire from	`/success?order=<uuid>` and `/orders/<uuid>`
Event id shape	`biz_k1QVwJt74iS6zP:purchase_<order uuid>`

The event id is already derived from the order uuid, which is exactly right and carries straight over to the server side version below. That part of the work is done.

3. Why this still needs to move

These events are fired from the browser. That was established on your own data rather than assumed, by asking a question a server side install cannot answer the same way: **did the buyer also register a page view at the moment of the purchase?**

Every one of the purchasing visitors did. 18 of 18 distinct buyers appear in the page view stream, and 32 of the 33 purchases have a page view from the same person within sixty seconds. An event raised by a server has no browser session attached to it and does not produce that pattern.

> **What this costs you, concretely.** A browser fire reports a sale only if the buyer's tab reaches the confirmation page and runs the script. It reports nothing when the customer closes the tab on the payment screen, when a content blocker or privacy browser stops the request, or when the payment is confirmed asynchronously minutes later. Those orders are real revenue that your ad reporting never sees, so the campaigns that produced them are optimizing against an incomplete picture.

4. What to build

One server side call, made from wherever your code already knows an order is paid for. The contract is:

```
POST https://api.whop.com/api/v1/events
Authorization: Bearer <your Whop API key>
Content-Type: application/json

{
  "event_name": "whop_purchase",
  "company_id": "biz_k1QVwJt74iS6zP",
  "event_id":   "<stable unique id for this order>",
  "value":      129.99,
  "currency":   "usd",
  "url":        "https://www.primescientificlab.com/...",
  "user":    { "anonymous_id": "<_wuid cookie>", "email": "buyer@example.com" },
  "context": { "user_agent": "<the buyer's user agent>" }
}
```

Note that on the **send** side the event name goes in `event_name`. On the read side the same event comes back as `pixel.custom` with the name in `custom_name`. That asymmetry is Whop's, not a mistake in this document.

Because your event ids already follow `purchase_<order uuid>`, keeping that exact shape means the server side events line up with the browser ones you already have, and running both briefly during the switchover will not double count.

4a. The attribution join, which is the part that is easy to miss

A server side event with no `user.anonymous_id` is a sale Whop cannot connect to the ad that produced it. It will be counted as revenue and attributed to nothing. The value comes from the `_wuid` cookie that the Whop pixel already sets in the buyer's browser, so it has to be captured in the browser and carried through to the server.

```
/* Runs in the browser at checkout, BEFORE the order is submitted. */
/* _wuid is the visitor id the Whop pixel already sets on your site. */
/* Without it the server event cannot be joined to the ad click, and */
/* the sale reports as unattributed. */
function whopWuid() {
  const m = document.cookie.match(/(?:^|;\s*)_wuid=([^;]+)/);
  return m ? decodeURIComponent(m[1]) : null;
}

/* Persist it ON THE ORDER RECORD, alongside the order id, so the */
/* server still has it when the payment confirms minutes later. */
orderPayload.whop_wuid = whopWuid();
```

> **Capture it at checkout, not at confirmation.** If the payment confirms after the customer has closed the tab, there is no browser left to read the cookie from. The whole point of moving this server side is that the tab is no longer required, and reading the cookie late gives that dependency straight back.

4b. The endpoint

```
/* app/api/whop-purchase/route.js  Next.js App Router on Vercel. */
/* Call this once from the code path that CONFIRMS an order, server side. */
/* Never from the browser: the customer's tab is not involved. */

/* Server env var only. Never NEXT_PUBLIC_. */
const WHOP_KEY = process.env.WHOP_API_KEY;
const COMPANY  = 'biz_k1QVwJt74iS6zP';

export async function POST(req) {
  const order = await req.json();

  /* event_id is what makes this safe to call more than once. Whop */
  /* de-duplicates on it, so a retry, a refresh or a replayed webhook */
  /* cannot double count the same order. */
  const body = {
    event_name: 'whop_purchase',
    company_id: COMPANY,
    event_id:   'purchase_' + order.id,
    value:      Number(order.total),
    currency:   'usd',
    url:        'https://www.primescientificlab.com/orders/' + order.id,
    user: {
      anonymous_id: order.whop_wuid || undefined,
      email:        order.email     || undefined
    },
    context: { user_agent: order.user_agent || undefined }
  };

  const r = await fetch('https://api.whop.com/api/v1/events', {
    method:  'POST',
    headers: {
      'Authorization': 'Bearer ' + WHOP_KEY,
      'Content-Type':  'application/json'
    },
    body: JSON.stringify(body)
  });

  if (!r.ok) {
    /* Log it and retry out of band. Do NOT fail the customer's order */
    /* because a tracking call failed. */
    console.error('whop purchase post failed', r.status, await r.text());
  }
  return new Response(null, { status: 204 });
}
```

> **Keep the API key server side.** On Vercel that means a plain environment variable, not one prefixed `NEXT_PUBLIC_`. A key exposed to the browser can be used by anyone to write events into your account.

5. How to confirm it works

Place one real order, then read the account back:

```
curl -s -H "Authorization: Bearer $WHOP_API_KEY" \\
  "https://api.whop.com/api/v1/events?company_id=biz_k1QVwJt74iS6zP&from=2026-09-
07T00:00:00Z&to=2026-09-08T00:00:00Z"
```

You are looking for a row with `custom_name: "whop_purchase"` carrying the order's real total in `total_usd_amount`. Two further checks separate a real server side install from a browser one:

1. **Send it twice.** Post the same `event_id` again. The event count must not go up. If it does, the idempotency key is not being set from anything stable.
2. **Kill the tab.** Complete a checkout and close the browser before the confirmation page renders. The purchase event must still arrive. This is the entire reason for the work, and it is the one test a browser install cannot pass.

> Whop's event stream lags ingestion by a few minutes, and an empty read taken too soon is not evidence of a failure. If a window looks empty, widen it to several days before concluding anything. A single empty response has produced more than one false alarm on this platform.